--bootstrap-server kafka:9092 --describe --topic logs 4. Spark Kafka connector és közös kód Spark 4.0.1 és Scala 2.13 esetén a Kafka connector koordinátája: org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.1 A connectornek elérhetőnek kell lennie a Spark driver és executor környezetében. Docker hálózaton belül a Kafka címe kafka:9092. from pyspark.sql import SparkSession import pyspark.sql.functions as F import pyspark.sql.types as T spark = (SparkSession.builder .appName("BigDataKafkaGyak") .getOrCreate()) spark.sparkContext.setLogLevel("WARN") KAFKA_BOOTSTRAP = "kafka:9092" def read_kafka(topic): return (spark.readStream .format("kafka") .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP) .option("subscribe", topic) .option("startingOffsets", "latest") .load()) def json_stream(topic, schema): raw = read_kafka(topic) return (raw.select(F.from_json(F.col("value").cast("string"), schema).alias("data")) .select("data.")) A Kafka value mezője bináris, ezért JSON feldolgozás előtt stringgé alakítjuk. 5. Sémák log_schema = T.StructType([T.StructField("timestamp", T.StringType()), T.StructField("srcip", T.StringType()), T.StructField("dstip", T.StringType()), T.StructField("method", T.StringType()), T.StructField("url", T.StringType()), T.StructField("status", T.IntegerType())]) sensor_schema = T.StructType([T.StructField("sensor_id", T.StringType()), T.StructField("timestamp", T.StringType()), T.StructField("type", T.StringType()), T.StructField("value", T.DoubleType())]) tweet_schema = T.StructType([T.StructField("user", T.StringType()), T.StructField("timestamp", T.StringType()), T.StructField("text", T.StringType()), T.StructField("hashtag", T.StringType())]) coordinate_schema = T.StructType([T.StructField("phone_id", T.StringType()), T.StructField("timestamp", T.StringType()), T.StructField("lat", T.DoubleType()), T.StructField("lon", T.DoubleType())]) 6. RDD és Structured Streaming – minta 6.1. RDD mikrobatch feldolgozás def process_batch(batch_df, batch_id): rdd = batch_df.rdd result = (rdd .take(2)) print("batch:", batch_id, result) query = (stream_df.writeStream .foreachBatch(process_batch) .outputMode("append") .start()) foreachBatch() minden mikrobatch-re meghívja a függvényt. Az RDD műveletek csak az adott batch rekordjait dolgozzák fel. A collect() és a takeOrdered() a driverre gyűjt eredményt, ezért csak kis oktatási kimenetre használd. 6.2. Natív Structured Streaming stream_df = json_stream("logs", log_schema) result = stream_df.count() query = (result.writeStream .format("console") .outputMode("complete") .start()) 7. Feladat 1 – URL-statisztika (logs) Mely URL-eket kérik le a legtöbbször? RDD-megoldás Structured Streaming megoldás 8. Feladat 2 – Támadásdetektálás (logs) 30 másodperces ablakban egy forrás IP legalább 5 különböző cél IP-t ér el. RDD-megoldás – mikrobatch szintű közelítés Structured Streaming megoldás 9. Feladat 3 – Szenzorstatisztika (sensors) Szenzoronként számíts átlagot, minimumot, maximumot és mérésszámot. RDD-megoldás Structured Streaming megoldás – 1 perces ablak 10. Feladat 4 – Tűzriasztás (sensors) Riasztás, ha egy hőmérséklet-szenzor 30 másodperces ablakban legalább háromszor 80 °C fölötti értéket mér. RDD-megoldás Structured Streaming megoldás 11. Feladat 5 – Hashtag-elemzés (tweets) RDD-megoldás Structured Streaming megoldás 12. Feladat 6 – Hangulatelemzés (tweets) Egyszerű lexikon: pozitív szó +1, negatív szó -1, egyéb szó 0. positive_words = {"love", "great", "amazing", "happy", "good", "excellent"} negative_words = {"bad", "terrible", "slow", "hate", "failed", "awful"} RDD-megoldás Structured Streaming megoldás 13. Feladat 7 – Térbeli cellák (coordinates) Oszd fel a koordinátákat rácscellákra, majd számold meg, hány különböző telefon található az egyes cellákban. RDD-megoldás Structured Streaming megoldás A fok alapú rács csak oktatási közelítés: a hosszúsági fokok távolsága a szélességtől függ. Éles térinformatikai rendszerben vetületi koordinátát vagy H3/S2 indexet használj. 14. Feladat 8 – Antennaelemzés (coordinates) Rendelj minden telefonkoordinátához legközelebbi cellaantennát. A kis antenna-referenciatáblát broadcastoljuk. 14.1. Antenna tábla és broadcast antenna_schema = T.StructType([T.StructField("antenna_id", T.StringType()), T.StructField("lat", T.DoubleType()), T.StructField("lon", T.DoubleType())]) antennas = spark.createDataFrame([("A1", 47.49790, 19.04020), ("A2", 47.50100, 19.05500), ("A3", 47.49200, 19.05000), ("A4", 47.50500, 19.03000)], antenna_schema) antenna_data = [r.asDict() for r in antennas.collect()] broadcast_antennas = spark.sparkContext.broadcast(antenna_data) RDD-megoldás Structured Streaming megoldás 15. Streaming query-k kezelése # Egy lekérdezés leállítása query.stop() # Aktív lekérdezések listázása for q in spark.streams.active: print(q.id, q.status) # Minden aktív lekérdezés leállítása for q in spark.streams.active: q.stop() A startingOffsets="latest" az újonnan érkező rekordoktól indítja a fogyasztást. Az earliest a topic elejéről olvas. Újraindításnál a checkpoint és az offsetek viselkedését is vizsgáld meg.*

Response: Big Data & Apache Spark Vizsga-összefoglaló (HDFS, MapReduce, RDD, DataFrame, Streaming, Kafka)

Ez a tananyag a teljes Big Data kurzus gyakorlati és elméleti anyagát dolgozza fel a vizsgára való felkészüléshez: a HDFS/MapReduce alapoktól kezdve az RDD és DataFrame API-n át a Structured Streaming és Kafka integrációig.

1. HDFS és MapReduce Alapok

1.1. HDFS Architektúra és Működés

A HDFS (*Hadoop Distributed File System*) nagy adathalmazok elosztott tárolását biztosítja szimpla hardverelemeken.

- **NameNode (Mester csomópont):** A fájlrendszer metaadatait kezeli memória-alapon (könyvtárstruktúra, fájlok, blokkok azonosítói és azok fizikai helye a DataNode-okon). Mint egy könyvtári katalógus.
- **DataNode (Munkás csomópont):** A tényleges adatblokkokat tárolja a helyi lemezein és kiszolgálja az olvasási/írási kéréseket.
- **HDFS Blokk:** A nagy fájlok rögzített méretű szeletekre (blokkokra) vannak vágva.
  - *Éles környezeti alapértelmezés:* 128 MB (vagy 256 MB).
  - *Oktatási Docker környezet:* 1 MB (hogy kisebb fájlokon is látszódjon a blokkokra bontás).
- **Replikáció:** Minden blokk több DataNode-on is el van tárolva (alapértelmezetten 3-szoros replikáció). Biztosítja a hibatűrést: ha egy DataNode kiesik, nem történik adatvesztés.

1.2. Legfontosabb HDFS CLI Parancsok

```
# Könyvtárstruktúra listázása
hdfs dfs -ls /
hdfs dfs -ls /data/raw/sensors

# Könyvtár létrehozása (-p: szülőkönyvtárakat is létrehozza)
hdfs dfs -mkdir -p /data/lab01

# Fájl feltöltése a helyi/konténer fájlrendszerből a HDFS-be (-f: felülírás)
hdfs dfs -put -f /course-data/historical/sensors10.json /data/raw/sensors/

# Fájl feltöltése egyedi blokkmérettel (pl. 64 MB)
hdfs dfs -D dfs.blocksize=67108864 -put /course-data/historical/sensors100.json /data/lab01/
```

```
# Fájl elejének megtekintése
hdfs dfs -head /data/raw/sensors/sensors10.json

# Tárhelyhasználat ellenőrzése (-h: emberileg olvasható formátum)
hdfs dfs -du -h /data/raw/sensors

# Cluster általános állapota (Live Node-ok, szabad/használt hely)
hdfs dfsadmin -report

# Fájl blokkjainak és fizikai helyének vizsgálata (FSCK)
hdfs fsck /data/lab01/sensors100.json -files -blocks -locations
```

Gyakorlati tapasztalat / Vizsgakérdés:

- **Tárhelyigény:** Kétszeres replikáció esetén egy 100 MB-os fájl $100 \times 2 = 200\ \text{MB}$ fizikai tárhelyet foglal el a HDFS-ben.
- **DataNode kiesése:** A NameNode szívverés (*heartbeat*) üzenetek alapján érzékeli a node kiesését. Ha a kiesett node-on lévő blokkok megvannak más node-on is, az adatok továbbra is elérhetőek maradnak. A replikációs szám csökkenése miatt a blokkok `under-replicated` állapotba kerülnek, amit a NameNode automatikusan helyreállít a meglévő replikák másolásával.

2. PySpark RDD (Resilient Distributed Dataset)

Az RDD a Spark alapszintű, immutábilis, elosztott adatszerkezete.

2.1. Spark Kontextus és Létrehozás

```
from pyspark import SparkConf, SparkContext

# SparkContext inicializálása (csak egyszer futtatható!)
conf = SparkConf().setMaster("spark://spark-master:7077").setAppName("ExamPrep")
sc = SparkContext(conf=conf)

# 1. Kollekcióból RDD (parallelize)
rdd_num = sc.parallelize(range(20), numSlices=4)

# 2. HDFS fájlból RDD (textFile)
rdd_text = sc.textFile("hdfs://namenode:8020/data/raw/sensors/sensors10.json")
```

2.2. Transzformációk vs. Akciók

- **Transzformációk (Lusta / Lazy evaluation):** Nem futtatnak számítást azonnal, csak leíró gráfot (DAG) építenek. Pl.: `map`, `filter`, `flatMap`, `reduceByKey`, `groupByKey`, `sortBy`, `distinct`, `join`, `coalesce`, `repartition`.
- **Akciók (Action):** Elindítják a tényleges kiszámítást a Spark klaszteren és eredményt adnak vissza a Drivernek vagy fájlba írnak. Pl.: `collect`, `take`, `count`, `first`, `reduce`, `fold`, `aggregate`, `saveAsTextFile`.

2.3. RDD Függvények Összefoglalása

Függvény	Típus	Leírás & Példa
`map(func)`	Transzformáció	1-to-1 transzformáció minden elemre. `rdd.map(lambda x: x * 2)`
`flatMap(func)`	Transzformáció	1-to-N transzformáció, a kapott listákat "kilapítja". `rdd.flatMap(lambda line: line.split(","))`
`filter(func)`	Transzformáció	Kiszűri a feltételnek nem megfelelő elemeket. `rdd.filter(lambda x: x % 2 == 0)`
`distinct()`	Transzformáció	Eltávolítja a duplikált elemeket.
`reduceByKey(func)`	Transzformáció	**Ajánlott!** Kulcsonként összevonja az értékeket. Mapper oldalon elő-összesít (combine), így kevés a network shuffle. `pairs.reduceByKey(lambda a, b: a + b)`
`groupByKey()`	Transzformáció	**Lassú!** Kulcsonként egy csoportba gyűjti az összes értéket elő-összesítés nélkül. Nagy adatnál OOM-et (Out Of Memory) okozhat.

Függvény	Típus	Leírás & Példa
`mapValues(func)`	Transzformáció	Csak a `(kulcs, érték)` párok érték részét alakítja át, a kulcs változatlan marad. `pairs.mapValues(lambda v: v + 10)`
`sortBy(keyfunc, asc)`	Transzformáció	Rendezi az RDD-t. `rdd.sortBy(lambda x: x[1], False)` (csökkenő)
`join(other)`	Transzformáció	Belső (inner) join két kulcs-érték RDD között. Létezik: `leftOuterJoin`, `rightOuterJoin`, `fullOuterJoin`.
`repartition(N)`	Transzformáció	Újraparticionál N partícióra. Teljes shuffle-t végez (növelésre és csökkentésre is jó).
`coalesce(N)`	Transzformáció	Partíciók számának csökkentése shuffle nélkül (hatékonyabb, mint a repartition).
`cache()`	Művelet	A memóriában tárolja az RDD-t az első Action lefutása után.
`glom()`	Transzformáció	Visszaadja a partíciók tartalmát listákként: `rdd.glom().collect()`
`collect()`	Akció	A teljes RDD-t a Driver memóriájába gyűjti (csak kis adatnál szabad!).
`take(n)`	Akció	Az első n elemet adja vissza listaként.
`reduce(func)`	Akció	Az összes elemet egyetlen értékké vonja össze. `rdd.reduce(lambda a, b: a + b)`
`aggregate(zero, seqOp, combOp)`	Akció	Összetett aggregáció kezdőértékkel (pl. átlag vagy min-max egyszerre).

3. PySpark DataFrame API és Spark SQL

A DataFrame egy névvel ellátott oszlopokba szervezett, elosztott adattábla (a Catalyst Optimizer optimalizálja a végrehajtását).

3.1. SparkSession és Beolvasás/Kiírás

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, expr, avg, count, sum, min, max, desc, round, to_date
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

spark = SparkSession.builder.appName("ExamApp").getOrCreate()

# Explicit séma megadása
schema = StructType([
    StructField("sensor_id", StringType(), False),
    StructField("timestamp", StringType(), True),
    StructField("value", DoubleType(), True)
])

# CSV olvasása
df_csv = spark.read.format("csv") \
    .option("header", True) \
    .option("inferSchema", True) \
    .load("hdfs://namenode:8020/data/raw/sensors/location")

# JSON olvasása
df_json = spark.read.json("hdfs://namenode:8020/data/raw/sensors/sensors100.json")

# Kiírás HDFS-re CSV-ben (több part-fájl keletkezik)
df_csv.write.mode("overwrite").option("header", True).csv("hdfs://namenode:8020/output/loc")

# Kiírás egyetlen CSV fájlba (coalesce 1)
df_csv.coalesce(1).write.mode("overwrite").option("header", True).csv("hdfs://namenode:8020/output/loc_single")
```

3.2. Legfontosabb DataFrame Műveletek

```
# 1. Oszlopok kiválasztása és kifejezések
df.select("sensor_id", "value").show(5)
df.selectExpr("sensor_id", "concat(location, ' ', sensor_type) as loc_sensor").show()

# 2. Szűrés (filter / where teljesen ekvivalens)
```

```
df.filter((col("location") == "Buda") & (col("value") > 20.0)).show()
df.where(~(col("location") == "Airport")).show()  # NOT operátor: ~

# 3. Rendezés
df.orderBy(col("value").desc()).show(10)

# 4. Egyedi értékek
df.select("location").distinct().show()

# 5. Joinek
# Tipusok: "inner", "left", "right", "full"
df_joined = df.join(dfLoc, df.location == dfLoc.location, how="left").drop(dfLoc.location)
# Ha az oszlopnév azonos mindkét táblában:
df_joined = df.join(dfLoc, on="location", how="left")

# 6. Csoportosítás és aggregáció
df.groupBy("sensor_type").agg(
    count("*").alias("cnt"),
    round(avg("value"), 2).alias("avg_val"),
    max("value").alias("max_val")
).filter(col("avg_val") > 15).orderBy(desc("cnt")).show()

# 7. Temp View és Spark SQL
df.createOrReplaceTempView("sensor_table")
res_sql = spark.sql("""
    SELECT sensor_type, COUNT(*) as cnt, AVG(value) as avg_val
    FROM sensor_table
    WHERE location != 'Airport'
    GROUP BY sensor_type
    HAVING avg_val > 10
    ORDER BY avg_val DESC
""")

# 8. Végrehajtási terv vizsgálata
res_sql.explain()
```

4. Spark Structured Streaming & Kafka Integráció

A **Structured Streaming** a Spark legújabb, streaming feldolgozómotorja, amely a DataFrame API-ra épül.

4.1. Alapfogalmak

- **Processing Time vs. Event Time:**
  - *Processing/Ingest Time:* Amikor a Spark feldolgozza a rekordot.
  - *Event Time:* Az esemény saját rekordjában lévő időbélyeg (pl. mikor mérte a szenzor az adatot).
- **Watermark:** Megadja, hogy a rendszer mennyi késést engedélyez az eseményeknél (*Event Time* alapon), mielőtt lezárja az ablakot és kidobja a régi állapotot a memóriából.
  - `.withWatermark("timestamp_col", "10 seconds")`
- **Output Mode-ok:**
  - `append`: Csak az új, véglegesen lezárt eredményeket írja ki (kötelező vízjel használata ablakos aggregációnál).
  - `update`: Azokat a sorokat írja ki a konzolra/storage-ra, amelyek frissültek a legutóbbi trigger óta.
  - `complete`: Minden egyes trigger alkalmával a teljes eredménytáblát újra kiírja.

4.2. Kafka Beolvasás és JSON Feldolgozás

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
import pyspark.sql.types as T

spark = SparkSession.builder \
    .appName("KafkaStreaming") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.1") \
    .getOrCreate()

# Kafka Stream Olvasás
kafka_raw = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "sensors") \
    .option("startingOffsets", "latest") \
    .load()
```

```
# A Kafka 'value' mezője bináris, stringgé castingoljuk és JSON-ként parse-oljuk
sensor_schema = T.StructType([
    T.StructField("sensor_id", T.StringType()),
    T.StructField("timestamp", T.StringType()),
    T.StructField("type", T.StringType()),
    T.StructField("value", T.DoubleType())
])

parsed_stream = kafka_raw.select(
    F.from_json(F.col("value").cast("string"), sensor_schema).alias("data")
).select("data.*")
```

4.3. Ablakos Aggregáció Streamingben (Fix & Csúszó ablak)

```
# Csúszó ablakos riasztás (10s ablak, 5s csúszás) vízjellel
alert_stream = parsed_stream \
    .withColumn("event_time", F.to_timestamp("timestamp")) \
    .withWatermark("event_time", "10 seconds") \
    .filter(F.col("type") == "temperature") \
    .groupBy(
        F.window(F.col("event_time"), "10 seconds", "5 seconds"),
        F.col("sensor_id")
    ) \
    .agg(
        F.sum(F.when(F.col("value") >= 80.0, 1).otherwise(0)).alias("high_cnt"),
        F.max("value").alias("max_temp")
    ) \
    .where("high_cnt >= 3")

# Kiírás Konzolra
query = alert_stream.writeStream \
    .format("console") \
    .outputMode("update") \
    .option("truncate", "false") \
    .start()
```

4.4. Streaming Mikrobatch RDD Feldolgozás (`foreachBatch`)

Ha a streaming adatot tetszőleges RDD kód segítségével akarjuk feldolgozni mikrobatchenként:

```
def process_microbatch(batch_df, batch_id):
    print(f"--- Batch ID: {batch_id} ---")
    # DataFrame átalakítása RDD-vé
    rdd = batch_df.rdd

    # Standard RDD műveletek
    top3 = rdd.map(lambda r: (r.sensor_id, r.value)) \
              .reduceByKey(lambda a, b: max(a, b)) \
              .sortBy(lambda x: x[1], False) \
              .take(3)

    for item in top3:
        print(item)

# foreachBatch indítása
query = parsed_stream.writeStream \
    .foreachBatch(process_microbatch) \
    .outputMode("append") \
    .start()
```

5. Típusfeladatok és Megoldások (Gyakorló Mintafeladatok)

Vizsgafeladat 1: RDD – Leggyakoribb elemek és párok

Feladat: Olvassuk be a vásárlási adatokat, alakítsuk kisbetűssé, és keressük meg a 3 leggyakoribb terméket!

```
# Bemenet: "Kenyer,Tej,cukor\nTej,KENYER"
lines = sc.textFile("hdfs://namenode:8020/data/vasarlasok.txt")

result = lines.flatMap(lambda line: line.split(",")) \
              .map(lambda word: word.strip().lower()) \
              .map(lambda word: (word, 1)) \
              .reduceByKey(lambda a, b: a + b) \
              .sortBy(lambda x: x[1], False) \
              .take(3)

print(result) # Output: [('kenyer', 108), ('cukor', 68), ('tej', 64)]
```

Vizsgafeladat 2: RDD – Napi Átlaghőmérséklet Számítás

Feladat: Számítsd ki az egyes napok átlagos hőmérsékletét az `Airport` helyszínre JSON formátumú szenzoradatokból!

```
import json

rdd_json = sc.textFile("hdfs://namenode:8020/data/raw/sensors/sensors100.json")

def parse_and_filter(line):
    data = json.loads(line)
    if data.get("location") == "Airport" and data.get("sensor_type") == "temperature":
        date = data["timestamp"].split("T")[0]
        val = float(data["value"])
        return [(date, (val, 1))]
    return []

daily_avg = rdd_json.flatMap(parse_and_filter) \
    .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])) \
    .mapValues(lambda x: round(x[0] / x[1], 2)) \
    .sortBy(lambda x: x[0]) \
    .collect()

print(daily_avg)
```

Vizsgafeladat 3: DataFrame – Max/Min szűrés és Összekapcsolás (Join)

Feladat: Kapcsold össze a menü és kategória táblát. Számold ki kategóriánként az átlagos kalóriát (kerekítve), hagyd figyelmen kívül a `'Beverages'` és `'Coffee & Tea'` kategóriákat, és rendezd csökkenő sorrendbe!

```
from pyspark.sql.functions import col, avg, round, desc

menu = spark.read.option("header", True).option("inferSchema", True).csv("menu.csv")
menuCat = spark.read.option("header", True).option("inferSchema", True).csv("menuCategory.csv")

res = menu.join(menuCat, col("Category") == col("Id")) \
    .where(~col("Name").isin(["Beverages", "Coffee & Tea"])) \
    .groupBy("Name") \
    .agg(round(avg("Calories")).alias("avgCal")) \
    .orderBy(desc("avgCal"))

res.show(10, truncate=False)
```

Vizsgafeladat 4: Streaming & Kafka – Támadásdetektálás (Sok cél IP egy forrásból)

Feladat: Detektáld, ha 30 másodperces ablakban egyetlen `srcip` legalább 5 különböző `dstip`-re küld kérést a weblog streaming adatokból!

```
from pyspark.sql import functions as F

# Használjuk a korábban definiált log_schema-t
logs_stream = read_kafka("logs") # Saját helper függvény

parsed_logs = logs_stream.select(
    F.from_json(F.col("value").cast("string"), log_schema).alias("d")
).select("d.*")

attack_detector = parsed_logs \
    .withColumn("event_time", F.to_timestamp("timestamp")) \
    .withWatermark("event_time", "30 seconds") \
    .groupBy(
        F.window("event_time", "30 seconds"),
        "srcip"
    ) \
    .agg(F.countDistinct("dstip").alias("distinct_dst_count")) \
    .filter(F.col("distinct_dst_count") >= 5)

query = attack_detector.writeStream \
    .format("console") \
    .outputMode("update") \
    .start()
```

Vizsgafeladat 5: Broadcast Join Streamingben (Telefon Antennakeresés)

Feladat: Kapcsold össze a telefonok streaming koordinátáit egy statikus referencia antenna táblával Broadcast segítségével, és határozd meg a legközelebbi antennát!

```
# Statikus antenna tábla
antenna_schema = T.StructType([
    T.StructField("antenna_id", T.StringType()),
    T.StructField("lat", T.DoubleType()),
```

```
    T.StructField("lon", T.DoubleType())
])
antennas_df = spark.createDataFrame([
    ("A1", 47.49790, 19.04020),
    ("A2", 47.50100, 19.05500)
], antenna_schema)

# Streaming koordináták
coords_stream = parsed_coord_stream.alias("c")
antennas_bcast = F.broadcast(antennas_df).alias("a")

# CrossJoin Broadcast-olt táblával (mert nincs közös kulcs, távolságot számolunk)
nearest_antenna = coords_stream.crossJoin(antennas_bcast) \
    .withColumn("dist_sq", F.pow(F.col("c.lat") - F.col("a.lat"), 2) + F.pow(F.col("c.lon") - F.col("a.lon"), 2)) \
    .withColumn("event_time", F.to_timestamp("c.timestamp")) \
    .withWatermark("event_time", "10 seconds") \
    .groupBy(F.window("event_time", "10 seconds"), "c.phone_id") \
    .agg(F.min(F.struct("dist_sq", "a.antenna_id")).alias("min_struct")) \
    .select("window", "phone_id", "min_struct.antenna_id", "min_struct.dist_sq")

query = nearest_antenna.writeStream.format("console").outputMode("append").start()
```

6. Gyors Ellenőrző Lista a Vizsgához

- **MapReduce / HDFS:**
  - Szétválik a tárolás (HDFS) és a számítás (Spark/MapReduce).
  - `reduceByKey` mindig jobb, mint a `groupByKey`, mert a Map oldalon végrehajtja a *Combiner* lépést.
- **RDD vs DataFrame:**
  - RDD: Típusbiztos, alacsony szintű, de nincs automatikus Catalyst optimalizáció.
  - DataFrame: Strukturált, sémával rendelkező, Catalyst Optimizer által szuper-optimális végrehajtási tervvé alakul.
- **PySpark Szintaxis apróságok:**
  - Szűrésnél összetett feltételnél **kötelező a zárójel** és a bitenkénti operátor: `(df["a"] == 1) & (df["b"] > 2)`.
  - Tagadás: `~(df["a"] == 1)`.
  - `filter()` és `where()` teljesen ugyanaz DataFrame-nél.
- **Streaming vizsgakérdések:**
  - `watermark`: megelőzi az állapottábla megnövekedését és korlátozza a késve érkező adatok várakozását.
  - `foreachBatch`: lehetővé teszi, hogy a Structured Streaming mikrobatcheit RDD API-val vagy tetszőleges külső könyvtárral (pl. SQL DB írás) dolgozzuk fel.
  - `outputMode("append")`: csak végleges (lezárt ablakú) adatot ír ki; `update`: csak a változottat; `complete`: az egész táblát újra.
